

### 作业三

一、 考虑以下两个计算 1 到 N 之间所有整数平方和的 Pthreads 程序片段。假设  $N = 10^9$ ，线程数 `thread_count = 8`

代码段 A:

```
void* Square_Sum_A(void* rank) {  
    long my_rank = (long) rank;  
  
    long long local_n = N / thread_count;  
  
    long long first = my_rank * local_n + 1;  
  
    long long last = first + local_n;  
  
    for (long long i = first; i < last; i++) {  
        pthread_mutex_lock(&mutex);  
  
        global_sum += i * i;  
  
        pthread_mutex_unlock(&mutex);  
    }  
  
    return NULL;  
}
```

代码段 B:

```
void* Square_Sum_B(void* rank) {  
    long my_rank = (long) rank;  
  
    long long local_n = N / thread_count;  
  
    long long my_sum = 0;  
  
    long long first = my_rank * local_n + 1;  
  
    long long last = first + local_n;  
  
    for (long long i = first; i < last; i++) {  
        my_sum += i * i;  
    }  
  
    pthread_mutex_lock(&mutex);
```

```

    global_sum += my_sum;

    pthread_mutex_unlock(&mutex);

    return NULL;
}

```

回答下列问题：

- (1) 指出代码段 A 中的临界区，并估算在  $N=10^9$  时，代码段 A 总共执行了多少次锁操作
- (2) 从操作系统调度的角度分析，为什么代码段 A 会导致 CPU 利用率虚高但实际计算进度缓慢
- (3) 在代码段 B 中，主线程在创建线程后立即调用 `pthread_join`（阻塞调用它的线程，直到指定的线程结束），请说明 `pthread_join` 在此处的作用

二、读写锁允许多个读者同时进入临界区，但写者必须互斥访问。考虑以下一个简单的、基于信号量实现的自定义读写锁逻辑：

```

int read_count = 0;

sem_t count_mutex; // 保护 read_count 的更新，初始化为 1
sem_t rw_mutex;    // 保护共享资源的写操作，初始化为 1

void* reader(void* arg) {
    sem_wait(&count_mutex);

    read_count++;

    if (read_count == 1)
        sem_wait(&rw_mutex); // 第一个读者锁定资源

    sem_post(&count_mutex);

    //执行读操作

    sem_wait(&count_mutex);
}

```

```

    read_count--;

    if (read_count == 0)
        sem_post(&rw_mutex); // 最后一个读者释放资源

    sem_post(&count_mutex);

    return NULL;
}

```

```

void* writer(void* arg) {
    sem_wait(&rw_mutex);

    // 执行写操作

    sem_post(&rw_mutex);

    return NULL;
}

```

回答下列问题：

- (1) 描述在该实现下，什么情况下写者可能会遭遇“饥饿”现象？
- (2) 该实现倾向于“读者”还是“写者”？如何避免对写者的饥饿问题，简要描述你的方案

三、某多线程程序中有两个共享数据结构 A 和 B，分别由互斥锁 `mutexA` 和 `mutexB` 保护。程序启动后有两个线程 T0 和 T1 并发执行，它们的加锁顺序如下：

线程 T0：先执行 `pthread_mutex_lock(&mutexA)`，再执行 `pthread_mutex_lock(&mutexB)`

线程 T1：先执行 `pthread_mutex_lock(&mutexB)`，再执行 `pthread_mutex_lock(&mutexA)`

假设系统中出现如下执行时序：

时刻 0：T0 成功获得 `mutexA`，T1 成功获得 `mutexB`

时刻 1: T0 试图获得 mutexB, T1 试图获得 mutexA

请回答下列问题:

- (1) 此时程序会发生什么现象? 请说明原因。
- (2) 为什么这种问题在线程程序中比较危险?
- (3) 给出一种简单可行的修改方法, 避免该问题再次发生。

四、阅读以下代码片段, 回答问题

```
#include <pthread.h>
```

```
#include <stdio.h>
```

```
int counter = 0; // 全局计数器
```

```
void* UnsafeIncrement(void* arg) {  
    for (int i = 0; i < 100000; i++) {  
        counter++;  
    }  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2;  
  
    pthread_create(&t1, NULL, UnsafeIncrement, NULL);  
    pthread_create(&t2, NULL, UnsafeIncrement, NULL);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    printf("Final Counter Value: %d\n", counter);  
    return 0;  
}
```

- (1) 理论上, 如果程序正确运行, counter 的最终值应该是多少? 为什么实际运行结果通常小于这个理论值?
- (2) 请指出 counter++ 操作在汇编层面通常包含哪三个步骤? 并解释这如何导致数据竞争。
- (3) 如何修改 UnsafeIncrement 函数, 使其线程安全

五、假设我们有一个多线程程序，用于统计两类事件发生的次数。两个线程分别只更新自己对应的计数器

```
#include <pthread.h>
#include <stdio.h>

// 定义一个结构体，包含两个计数器
typedef struct {
    long counter_a; // 线程 0 专门累加这个
    long counter_b; // 线程 1 专门累加这个
} CounterPair;

CounterPair counters;

void* ThreadFunc(void* arg) {
    long tid = (long)arg;
    // 模拟大量工作
    for (int i = 0; i < 1000000000; i++) {
        if (tid == 0) {
            counters.counter_a++; // 线程 0 操作
        } else {
            counters.counter_b++; // 线程 1 操作
        }
    }
    return NULL;
}

int main() {
    pthread_t t0, t1;
    // 创建两个线程
    pthread_create(&t0, NULL, ThreadFunc, (void*)0);
    pthread_create(&t1, NULL, ThreadFunc, (void*)1);

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    printf("A: %ld, B: %ld\n", counters.counter_a, counters.counter_b);
    return 0;
}
```

- (1) 为什么上述代码可能存在严重的伪共享问题，并解释一下为何在两个线程上操作不同的变量，却会导致性能急剧下降？
- (2) 为了消除伪共享，请修改 CounterPair 结构体的定义，利用填充技术

确保 `counter_a` 和 `counter_b` 位于不同的缓存行中。假设系统的缓存行大小为 64 字节。

- (3) 除了填充结构体外，还有另一种利用局部性原理的高性能方案，如何修改 `ThreadFunc` 函数的逻辑，让每个线程先在“私有”的内存空间进行累加，最后再合并结果，从而彻底避开伪共享？